

1. Nombres de variables: claros, coherentes y descriptivos

Buenas prácticas

- Usa **camelCase** para variables y funciones: `nombreAlumno`, `notaMedia`.
- Los nombres deben **indicar claramente** qué guardan o representan.
- Evita abreviaturas confusas (`tmp`, `n`, `x`, `a1...`).
- Prefiere nombres **en singular o plural** según el contenido (`usuario` vs `usuarios`).
- Sé coherente: usa el mismo estilo en todo el proyecto.

Ejemplo

```
//  Malo  
let a = 9.5;  
let n = "Carlos";
```

```
//  Bueno  
let notaFinal = 9.5;  
let nombreAlumno = "Carlos";
```

 Consejo: **Lee tu variable en voz alta**. Si suena natural en la frase “la variable `nombreAlumno` contiene...”, probablemente está bien nombrada.

2. Clases: representan entidades o conceptos

Buenas prácticas

- Usa **PascalCase** para nombres de clases: `Alumno`, `Producto`, `ReservaHotel`.
- Cada clase debe tener **una sola responsabilidad** (principio SRP de SOLID).
- Agrupa dentro de la clase solo los datos y comportamientos relacionados con esa entidad.

⚡ Ejemplo

// ❌ Clase genérica y confusa

```
class Utilidades {  
  guardar() {}  
  calcularMedia() {}  
  mostrarDatos() {}  
}
```

// ✅ Clase bien estructurada

```
class Alumno {  
  constructor(nombre, notas) {  
    this.nombre = nombre;  
    this.notas = notas;  
  }  
  
  calcularMedia() {  
    const suma = this.notas.reduce((a, b) => a + b, 0);  
    return suma / this.notas.length;  
  }  
}
```

🧠 Consejo: Si tu clase hace más de una cosa (guardar, calcular, mostrar...), divídela en varias más pequeñas y específicas.

⚙️ 3. Métodos: deben describir una acción (verbo + objeto)

✅ Buenas prácticas

- Usa **verbos** al inicio: `calcular`, `mostrar`, `actualizar`, `guardar`, `eliminar`.
- Un método debe **hacer solo una cosa**.
- Usa nombres en **camelCase**.
- No hagas métodos muy largos (idealmente menos de 20 líneas).

⚡ Ejemplo

// ❌ Malo

```
mostrar() { /* muestra muchas cosas a la vez */ }
```

```
//  Bueno
```

```
mostrarNombre() {}
```

```
mostrarNotas() {}
```

```
calcularPromedio() {}
```

 Regla de oro: **si un método necesita más de un “y” para describirlo, haz dos métodos.**

4. Funciones fuera de clases (funciones puras y helpers)

No todo debe ser una clase.

Las **funciones puras** (que no dependen del estado externo) son más fáciles de probar y entender.

Ejemplo

```
// Función pura (no depende de variables externas)
```

```
function calcularMedia(notas) {  
  return notas.reduce((a, b) => a + b, 0) / notas.length;  
}
```

```
// Función con efecto secundario (interactúa con el DOM)
```

```
function mostrarMensaje(mensaje) {  
  document.getElementById("info").textContent = mensaje;  
}
```

Consejo:

- Si una función **usa `this`**, probablemente pertenece a una clase.
 - Si no usa **`this`**, déjala fuera como **función utilitaria**.
-

5. Constantes y valores fijos

✓ Buenas prácticas

- Usa `const` para valores que **no cambian**.
- Usa `let` para valores que **sí cambian**.
- No uses `var` (ya está obsoleto).
- Usa `MAYÚSCULAS_SNAKE_CASE` para constantes globales.

⚡ Ejemplo

```
const MAX_NOTA = 10;  
let notaActual = 8.5;
```

🧩 6. Estructura recomendada de un archivo o módulo JS

```
// ① Constantes y configuraciones iniciales  
const MAX_NOTA = 10;  
  
// ② Clases  
class Alumno {  
  constructor(nombre, notas) {  
    this.nombre = nombre;  
    this.notas = notas;  
  }  
  
  calcularMedia() {  
    return this.notas.reduce((a, b) => a + b, 0) /  
    this.notas.length;  
  }  
}  
  
// ③ Funciones auxiliares (helpers)  
function mostrarEnPantalla(texto) {  
  document.getElementById("resultado").textContent = texto;  
}  
  
// ④ Código principal
```

```
const alumno = new Alumno("Lucía", [8, 9, 7, 10, 9]);  
mostrarEnPantalla(`Media: ${alumno.calcularMedia()}`);
```

7. Principios de código limpio en JavaScript

* Evita duplicar código

Si repites el mismo bloque, conviértelo en una función o método.

* Encapsula los datos

Usa `_nombre` o `#nombre` para propiedades privadas (en ES2022 o superior).

```
class Alumno {  
  #nombre;  
  constructor(nombre) {  
    this.#nombre = nombre;  
  }  
}
```

* Usa arrow functions solo cuando tenga sentido

Evita usarlas dentro de clases (pierden el `this` propio).

```
// ❌ Dentro de una clase:  
mostrarNombre = () => console.log(this.nombre); // puede fallar
```

```
// ✅ Mejor  
mostrarNombre() { console.log(this.nombre); }
```

* Comenta solo lo necesario

Los **nombres claros** sustituyen los comentarios.

Usa comentarios para explicar **por qué** haces algo, no **qué** hace.

8. Estandariza tu estilo de código

Usa herramientas de linting y formato automático:

Herramienta	Propósito
ESLint	Detecta errores de estilo y sintaxis
Prettier	Da formato uniforme al código
JSDoc	Documenta tus clases y métodos con comentarios estructurados

⚡ Ejemplo de JSDoc:

```
/**
 * Calcula la media de un conjunto de notas.
 * @param {number[]} notas - Array de calificaciones.
 * @returns {number} Media aritmética de las notas.
 */
function calcularMedia(notas) {
  return notas.reduce((a, b) => a + b, 0) / notas.length;
}
```

🎯 9. Resumen visual de buenas prácticas

Elemento	Convención	Ejemplo	Consejo
Clase	PascalCase	Alumno, Producto	Sustantivo, una sola responsabilidad
Método	camelCase	calcularMedia()	Verbo + objeto
Variable	camelCase	notaFinal, nombreAlumno	Descriptiva
Constante	MAYÚSCULAS_SNAKE_CASE	MAX_NOTAS	Solo si no cambia
Archivo	kebab-case	alumno-class.js	Nombres simples y coherentes

🧩 10. Bonus: patrones de organización recomendados

- **Módulos (ES Modules):**
Divide tu código en archivos (`import/export`) para evitar un solo archivo gigante.
- **Principio DRY (Don't Repeat Yourself):**
No repitas lógica. Extrae funciones reutilizables.
- **Principio KISS (Keep It Simple, Stupid):**
No compliques el código innecesariamente.
- **Principio YAGNI (You Aren't Gonna Need It):**
No implementes funciones que aún no necesitas.